

```

1 # ===== DETERMINISM HEADER =====
2 import os, random, gc
3
4 SEED = 445          # single source of truth
5 DETERMINISTIC = True      # flip to False for speed over strict repeatability
6 THREADS = 1 if DETERMINISTIC else max(1, os.cpu_count() // 2)
7
8 # Make numeric libs single-threaded to avoid nondeterministic reductions
9 os.environ["OMP_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
10 os.environ["OPENBLAS_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
11 os.environ["MKL_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
12 os.environ["VECLIB_MAXIMUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
13 os.environ["NUMEXPR_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
14
15 # Python & NumPy RNG
16 os.environ["PYTHONHASHSEED"] = str(SEED)
17 random.seed(SEED)
18 # =====
19
20 # Core libs
21 import pandas as pd
22 import numpy as np
23 from pathlib import Path
24
25 # Sklearn base imports
26 from sklearn.impute import SimpleImputer
27 from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
28 from sklearn.preprocessing import StandardScaler
29 from sklearn.pipeline import Pipeline
30 from sklearn.linear_model import LogisticRegression
31 from sklearn.metrics import (
32     roc_auc_score, roc_curve, precision_recall_curve, average_precision_score,
33     confusion_matrix, classification_report
34 )
35
36 # Model libraries
37 from xgboost import XGBClassifier
38 from catboost import CatBoostClassifier
39 from sklearn.ensemble import RandomForestClassifier
40
41 # Plotting
42 import matplotlib.pyplot as plt
43
44 # For reproducibility of random numbers in simulations
45 import hashlib
46
47 # ===== Load Data & Data Pre-processing
48 # =====
49 # --- file paths (edit TEST_PATH if needed) ---
50 # r"H:\git\ML-Credit-Risk\Dataset\cs-training.csv"
51 # r"D:\Github\ML-Credit-Risk\Dataset\GiveMeSomeCredit\cs-training.csv"
52 df = pd.read_csv(r"H:\git\ML-Credit-Risk\Dataset\cs-training.csv")
53
54 # drop index column if exists
55 df = df.drop(columns=["Unnamed: 0"], errors="ignore")
56
57 # keep ages 20–100
58 df = df[(df["age"] >= 20) & (df["age"] <= 100)].copy()
59
60 # MonthlyIncome: keep 0s, impute only NAs by age-group median ===
61 df["NoIncomeFlag"] = (df["MonthlyIncome"] == 0).astype("int8")
62
63 bins = [20, 30, 40, 50, 60, 70, 80, 90, 100]
64 labels = ["20-29", "30-39", "40-49", "50-59", "60-69", "70-79", "80-89", "90-100"]
65 df["age_group"] = pd.cut(df["age"], bins=bins, labels=labels, right=False)

```

```

66
67 median_by_group = (
68     df.loc[df["MonthlyIncome"] > 0]
69     .groupby("age_group", observed=False)["MonthlyIncome"]
70     .median()
71 )
72 mi_na_mask = df["MonthlyIncome"].isna()
73 df.loc[mi_na_mask, "MonthlyIncome"] = df.loc[mi_na_mask, "age_group"].map(median_by_group)
74 df["IncomeImputedFlag"] = mi_na_mask.astype("int8")
75 df = df.drop(columns=["age_group"])
76
77 # DebtRatio: coerce, remove inf, and cap per new rules (0-income=10, imputed=2, normal=10)
78 dr = pd.to_numeric(df["DebtRatio"], errors="coerce")
79 dr = dr.mask(np.isinf(dr), np.nan) # remove ±inf safely
80
81 # Masks from existing columns/flags
82 pos_income = df["MonthlyIncome"] > 0
83 was_imputed = df["IncomeImputedFlag"].eq(1) # MonthlyIncome was NA and then imputed by
84     median
85 zero_income = df["NoIncomeFlag"].eq(1) # MonthlyIncome == 0 (flag set before
86     imputation)
87 normal_income = pos_income & (~was_imputed) # positive and not imputed
88
89 # Caps
90 REALISTIC_CAP = 10.0
91 IMPUTED_CAP = 2.0
92
93 # Start from a clean series
94 dr_series = pd.Series(dr, index=df.index)
95
96 # Apply caps with lower bound at 0 (avoid negatives)
97 dr_series.loc[was_imputed] = np.clip(dr_series.loc[was_imputed], 0.0, IMPUTED_CAP)
98 dr_series.loc[zero_income] = np.clip(dr_series.loc[zero_income], 0.0, REALISTIC_CAP)
99 dr_series.loc[normal_income] = np.clip(dr_series.loc[normal_income], 0.0, REALISTIC_CAP)
100
101 # Fill any remaining NaNs with the median of observed values
102 dr_series = dr_series.fillna(np.nanmedian(dr_series.values))
103 df["DebtRatio"] = dr_series.astype(float)
104
105 # NumberOfDependents: flag missing, top-code, impute median, large-family flag
106 dep_na_mask = df["NumberOfDependents"].isna()
107 df["DependentsMissingFlag"] = dep_na_mask.astype("int8")
108
109 # top-code extremes at 5 before imputation
110 dep_series = df["NumberOfDependents"].copy()
111 dep_series = dep_series.where(dep_series.isna() | (dep_series <= 5), 5)
112 dep_median = int(dep_series.median(skipna=True))
113 dep_series = dep_series.fillna(dep_median).astype("int8")
114
115 df["NumberOfDependents"] = dep_series
116 df["LargeFamilyFlag"] = (df["NumberOfDependents"] >= 5).astype("int8")
117
118 # Number of Times Past Due Clean up
119 cols_past_due = [
120     "NumberOfTime30-59DaysPastDueNotWorse",
121     "NumberOfTime60-89DaysPastDueNotWorse",
122     "NumberOfTimes90DaysLate"
123 ]
124 for col in cols_past_due:
125     df[col] = pd.to_numeric(df[col], errors="coerce")
126     df[f"{col}_MissingFlag"] = df[col].isin([96, 98]).astype(int)
127     df[col] = df[col].replace({96: np.nan, 98: np.nan})
128     median_val = df[col].median(skipna=True)
129     df[col] = df[col].fillna(median_val)
130     df[col] = np.where(df[col] > 10, 10, df[col]).astype(int)

```

```

129
130 # ===== Split the dataset
    =====
131 TARGET = "SeriousDlqin2yrs"
132 X = df.drop(columns=[TARGET])
133 y = df[TARGET].astype(int)
134
135 # First split into train+val and test (stratified)
136 X_trainval, X_test, y_trainval, y_test = train_test_split(
137     X, y, test_size=0.2, stratify=y, random_state=SEED
138 )
139 X_train, X_val, y_train, y_val = train_test_split(
140     X_trainval, y_trainval, test_size=0.25, stratify=y_trainval, random_state=SEED
141 )
142
143 # ----- Persist splits so future runs use identical rows -----
144 split_dir = Path("./_splits"); split_dir.mkdir(exist_ok=True)
145 f_train = split_dir / "train_idx.npy"
146 f_val = split_dir / "val_idx.npy"
147 f_test = split_dir / "test_idx.npy"
148
149 if all(f.exists() for f in (f_train, f_val, f_test)):
150     # Rebuild splits from saved indices
151     idx_train = np.load(f_train, allow_pickle=False)
152     idx_val = np.load(f_val, allow_pickle=False)
153     idx_test = np.load(f_test, allow_pickle=False)
154     X_train, y_train = X.loc[idx_train], y.loc[idx_train]
155     X_val, y_val = X.loc[idx_val], y.loc[idx_val]
156     X_test, y_test = X.loc[idx_test], y.loc[idx_test]
157 else:
158     # First run: save the indices we just generated with the fixed random_state
159     np.save(f_train, X_train.index.values)
160     np.save(f_val, X_val.index.values)
161     np.save(f_test, X_test.index.values)
162
163
164 # ===== L1 Logistic Regression
    =====
165 pipe = Pipeline(steps=[
166     ("scaler", StandardScaler()),
167     ("clf", LogisticRegression(
168         penalty="l1",
169         solver="saga",
170         max_iter=5000,
171         class_weight="balanced",
172         random_state=SEED,
173         n_jobs=THREADS
174     ))
175 ])
176
177 cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
178
179 param_grid = {"clf__C": np.logspace(-3, 1, 16)}
180
181 grid = GridSearchCV(
182     estimator=pipe,
183     param_grid=param_grid,
184     scoring="roc_auc",
185     cv=cv,
186     n_jobs=1,
187     verbose=1
188 )
189
190 grid.fit(X_train, y_train)
191 best_model = grid.best_estimator_
192 print("Best C:", grid.best_params_["clf__C"])

```

Project code.py — generated 2025-08-15 17:05:47

```
193 print("CV ROC-AUC (mean):", grid.best_score_)
194
195 # Validate on VAL set
196 val_proba = best_model.predict_proba(X_val)[:, 1]
197 val_auc = roc_auc_score(y_val, val_proba)
198 val_prauc = average_precision_score(y_val, val_proba)
199
200 # KS statistic
201 fpr, tpr, thr = roc_curve(y_val, val_proba)
202 ks_vals = tpr - fpr
203 ks = np.max(ks_vals)
204 thr_ks = thr[np.argmax(ks_vals)]
205
206 print(f"Validation ROC-AUC: {val_auc:.4f}")
207 print(f"Validation PR-AUC : {val_prauc:.4f}")
208 print(f"Validation KS : {ks:.4f} @ threshold {thr_ks:.4f}")
209
210 # Choose threshold by max KS on validation
211 val_pred = (val_proba >= thr_ks).astype(int)
212 print("\nValidation Confusion Matrix:")
213 print(confusion_matrix(y_val, val_pred))
214 print("\nValidation Classification Report:")
215 print(classification_report(y_val, val_pred, digits=4))
216
217 # Retrain on TRAIN+VAL with best hyperparams, then evaluate on TEST
218 best_model_final = GridSearchCV(
219     estimator=pipe,
220     param_grid={"clf__C": [grid.best_params_["clf__C"]]},
221     scoring="roc_auc",
222     cv=cv,
223     n_jobs=-1
224 ).fit(pd.concat([X_train, X_val]), pd.concat([y_train, y_val])).best_estimator_
225
226 test_proba = best_model_final.predict_proba(X_test)[:, 1]
227 test_auc = roc_auc_score(y_test, test_proba)
228 test_prauc = average_precision_score(y_test, test_proba)
229 fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
230 ks_t = np.max(tpr_t - fpr_t)
231
232 # Use the threshold chosen on validation
233 test_pred = (test_proba >= thr_ks).astype(int)
234
235 print("\n=== TEST RESULTS (using validation KS threshold) ===")
236 print(f"Test ROC-AUC: {test_auc:.4f}")
237 print(f"Test PR-AUC : {test_prauc:.4f}")
238 print(f"Test KS : {ks_t:.4f}")
239 print("\nTest Confusion Matrix:")
240 print(confusion_matrix(y_test, test_pred))
241 print("\nTest Classification Report:")
242 print(classification_report(y_test, test_pred, digits=4))
243
244
245 # ===== XGBoost =====
246 X_train = X_train.astype(np.float32, copy=False)
247 X_val = X_val.astype(np.float32, copy=False)
248 X_test = X_test.astype(np.float32, copy=False)
249
250 # CV setup
251 cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
252 pos, neg = int((y_train == 1).sum()), int((y_train == 0).sum())
253 scale_pos_weight = neg / max(pos, 1)
254
255 pipe_xgb = Pipeline([
256     ("imp", SimpleImputer(strategy="median")),
257     ("xgb", XGBClassifier(
258         objective="binary:logistic",
```

```

259     eval_metric="auc",
260     tree_method="hist",
261     learning_rate=0.05,
262     n_estimators=600,
263     max_depth=6,
264     subsample=0.8,
265     colsample_bytree=0.8,
266     min_child_weight=1.0,
267     reg_lambda=1.0,
268     reg_alpha=0.0,
269     scale_pos_weight=scale_pos_weight,
270     n_jobs=THREADS,
271     random_state=SEED,
272     verbosity=0
273 ))
274 ])
275
276 # compact grid
277 param_grid_xgb = {
278     "xgb__n_estimators": [400, 800],
279     "xgb__max_depth": [4, 6],
280     "xgb__learning_rate": [0.05, 0.1],
281     "xgb__min_child_weight": [1.0, 3.0],
282 }
283
284 grid_xgb = GridSearchCV(
285     estimator=pipe_xgb,
286     param_grid=param_grid_xgb,
287     scoring="roc_auc",
288     cv=cv,
289     n_jobs=1,
290     verbose=1,
291     refit=True
292 )
293
294 # Fit CV
295 grid_xgb.fit(X_train, y_train)
296 best_xgb = grid_xgb.best_estimator_
297 print("Best params:", grid_xgb.best_params_)
298 print("CV ROC-AUC :", grid_xgb.best_score_)
299 gc.collect()
300
301 # Validate on VAL — get KS threshold
302 val_proba = best_xgb.predict_proba(X_val)[, 1]
303 val_auc = roc_auc_score(y_val, val_proba)
304 val_prauc = average_precision_score(y_val, val_proba)
305 fpr, tpr, thr = roc_curve(y_val, val_proba)
306 ks_vals = tpr - fpr
307 ks = float(np.max(ks_vals))
308 thr_ks = float(thr[np.argmax(ks_vals)])
309
310 print(f"Validation ROC-AUC: {val_auc:.4f}")
311 print(f"Validation PR-AUC : {val_prauc:.4f}")
312 print(f"Validation KS    : {ks:.4f} @ threshold {thr_ks:.4f}")
313
314 val_pred = (val_proba >= thr_ks).astype(int)
315 print("\nValidation Confusion Matrix:")
316 print(confusion_matrix(y_val, val_pred))
317 print("\nValidation Classification Report:")
318 print(classification_report(y_val, val_pred, digits=4))
319 del val_proba, fpr, tpr, thr, ks_vals
320 gc.collect()
321
322 # Refit on TRAIN+VAL with best hyperparams
323 best_xgb_final = grid_xgb.best_estimator_
324 X_trv = pd.concat([X_train, X_val], copy=False)

```

Project code.py — generated 2025-08-15 17:05:47

```
325 y_trv = pd.concat([y_train, y_val], copy=False)
326 best_xgb_final.fit(X_trv, y_trv)
327 del X_trv, y_trv
328 gc.collect()
329
330 # Final TEST evaluation (use validation KS threshold)
331 test_proba = best_xgb_final.predict_proba(X_test)[: , 1]
332 test_auc = roc_auc_score(y_test, test_proba)
333 test_prauc = average_precision_score(y_test, test_proba)
334 fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
335 ks_t = float(np.max(tpr_t - fpr_t))
336 test_pred = (test_proba >= thr_ks).astype(int)
337
338 print("\n=== TEST RESULTS (using validation KS threshold) ===")
339 print(f"Test ROC-AUC: {test_auc:.4f}")
340 print(f"Test PR-AUC : {test_prauc:.4f}")
341 print(f"Test KS : {ks_t:.4f}")
342 print("\nTest Confusion Matrix:")
343 print(confusion_matrix(y_test, test_pred))
344 print("\nTest Classification Report:")
345 print(classification_report(y_test, test_pred, digits=4))
346
347
348 # ===== CatBoost =====
349 X_train_cb = X_train.copy()
350 X_val_cb = X_val.copy()
351 X_test_cb = X_test.copy()
352
353 # Class weight for imbalance
354 pos = int((y_train == 1).sum())
355 neg = int((y_train == 0).sum())
356 class_weights = [1.0, neg / max(pos, 1)]
357
358 # Small param grid
359 param_grid_cb = [
360     {"depth": 6, "learning_rate": 0.05, "l2_leaf_reg": 3.0},
361     {"depth": 8, "learning_rate": 0.05, "l2_leaf_reg": 3.0},
362     {"depth": 6, "learning_rate": 0.1, "l2_leaf_reg": 3.0},
363 ]
364
365 best_cb = None
366 best_auc = -np.inf
367 best_params = None
368
369 # Simple manual CV loop for small grid
370 for params in param_grid_cb:
371     cb = CatBoostClassifier(
372         task_type="CPU",
373         iterations=2000,
374         early_stopping_rounds=100,
375         loss_function="Logloss",
376         eval_metric="AUC",
377         class_weights=class_weights,
378         random_seed=SEED,
379         thread_count=THREADS,
380         verbose=False,
381         **params
382     )
383     cb.fit(X_train_cb, y_train, eval_set=(X_val_cb, y_val), use_best_model=True)
384     val_proba = cb.predict_proba(X_val_cb)[: , 1]
385     val_auc = roc_auc_score(y_val, val_proba)
386     if val_auc > best_auc:
387         best_auc = val_auc
388         best_cb = cb
389         best_params = params
390
```

```

391 print("Best CatBoost params:", best_params)
392 print("Validation ROC-AUC:", best_auc)
393
394 # Compute validation PR-AUC and KS
395 val_proba = best_cb.predict_proba(X_val_cb)[: , 1]
396 val_prauc = average_precision_score(y_val, val_proba)
397 fpr, tpr, thr = roc_curve(y_val, val_proba)
398 ks_vals = tpr - fpr
399 ks = float(np.max(ks_vals))
400 thr_ks_cb = float(thr[np.argmax(ks_vals)])
401 print(f"Validation PR-AUC: {val_prauc:.4f}")
402 print(f"Validation KS : {ks:.4f} @ threshold {thr_ks_cb:.4f}")
403
404 val_pred = (val_proba >= thr_ks_cb).astype(int)
405 print("\nValidation Confusion Matrix:")
406 print(confusion_matrix(y_val, val_pred))
407 print("\nValidation Classification Report:")
408 print(classification_report(y_val, val_pred, digits=4))
409
410 # Retrain on TRAIN+VAL
411 cb_final = CatBoostClassifier(
412     task_type="CPU",
413     iterations=2000,
414     early_stopping_rounds=100,
415     loss_function="Logloss",
416     eval_metric="AUC",
417     class_weights=class_weights,
418     random_seed=42,
419     verbose=False,
420     **best_params
421 )
422 X_trv_cb = pd.concat([X_train_cb, X_val_cb], copy=False)
423 y_trv_cb = pd.concat([y_train, y_val], copy=False)
424 cb_final.fit(X_trv_cb, y_trv_cb, use_best_model=False)
425
426 # Test set evaluation
427 test_proba = cb_final.predict_proba(X_test_cb)[: , 1]
428 test_auc = roc_auc_score(y_test, test_proba)
429 test_prauc = average_precision_score(y_test, test_proba)
430 fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
431 ks_t = float(np.max(tpr_t - fpr_t))
432 test_pred = (test_proba >= thr_ks_cb).astype(int)
433
434 print("\n=== TEST RESULTS (CatBoost, using validation KS threshold) ===")
435 print(f"Test ROC-AUC: {test_auc:.4f}")
436 print(f"Test PR-AUC : {test_prauc:.4f}")
437 print(f"Test KS : {ks_t:.4f}")
438 print("\nTest Confusion Matrix:")
439 print(confusion_matrix(y_test, test_pred))
440 print("\nTest Classification Report:")
441 print(classification_report(y_test, test_pred, digits=4))
442 gc.collect()
443
444
445 # ===== Random Forest
446 X_train_rf = X_train.astype(np.float32, copy=False)
447 X_val_rf = X_val.astype(np.float32, copy=False)
448 X_test_rf = X_test.astype(np.float32, copy=False)
449
450 cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
451
452 pipe_rf = Pipeline([
453     ("imp", SimpleImputer(strategy="median")),
454     ("rf", RandomForestClassifier(
455         n_estimators=600,

```

```

456     max_depth=None,
457     max_features="sqrt",
458     min_samples_split=2,
459     min_samples_leaf=1,
460     class_weight="balanced_subsample",
461     n_jobs=max(1, os.cpu_count() // 2),
462     random_state=42
463 ))
464 ])
465
466 # compact grid
467 param_grid_rf = {
468     "rf__n_estimators": [400, 800],
469     "rf__max_depth": [None, 12],
470     "rf__min_samples_leaf": [1, 5],
471     "rf__max_features": ["sqrt", 0.5],
472 }
473
474 grid_rf = GridSearchCV(
475     estimator=pipe_rf,
476     param_grid=param_grid_rf,
477     scoring="roc_auc",
478     cv=cv,
479     n_jobs=1,
480     verbose=1,
481     refit=True
482 )
483
484 # Fit CV
485 grid_rf.fit(X_train_rf, y_train)
486 best_rf = grid_rf.best_estimator_
487 print("Best RF params:", grid_rf.best_params_)
488 print("CV ROC-AUC   :", grid_rf.best_score_)
489
490 # Validate — pick KS threshold
491 val_proba = best_rf.predict_proba(X_val_rf)[:, 1]
492 val_auc = roc_auc_score(y_val, val_proba)
493 val_prauc = average_precision_score(y_val, val_proba)
494 fpr, tpr, thr = roc_curve(y_val, val_proba)
495 ks_vals = tpr - fpr
496 ks = float(np.max(ks_vals))
497 thr_ks_rf = float(thr[np.argmax(ks_vals)])
498
499 print(f"Validation ROC-AUC: {val_auc:.4f}")
500 print(f"Validation PR-AUC : {val_prauc:.4f}")
501 print(f"Validation KS    : {ks:.4f} @ threshold {thr_ks_rf:.4f}")
502
503 val_pred = (val_proba >= thr_ks_rf).astype(int)
504 print("\nValidation Confusion Matrix:")
505 print(confusion_matrix(y_val, val_pred))
506 print("\nValidation Classification Report:")
507 print(classification_report(y_val, val_pred, digits=4))
508 del val_proba, fpr, tpr, thr, ks_vals
509 gc.collect()
510
511 # Refit on TRAIN+VAL with best hyperparams
512 best_rf_final = grid_rf.best_estimator_
513 X_trv_rf = pd.concat([X_train_rf, X_val_rf], copy=False)
514 y_trv_rf = pd.concat([y_train, y_val], copy=False)
515 best_rf_final.fit(X_trv_rf, y_trv_rf)
516 del X_trv_rf, y_trv_rf
517 gc.collect()
518
519 # Final TEST evaluation (use validation KS threshold)
520 test_proba = best_rf_final.predict_proba(X_test_rf)[:, 1]
521 test_auc = roc_auc_score(y_test, test_proba)

```


Project code.py — generated 2025-08-15 17:05:47

```
522 test_prauc = average_precision_score(y_test, test_proba)
523 fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
524 ks_t = float(np.max(tpr_t - fpr_t))
525 test_pred = (test_proba >= thr_ks_rf).astype(int)
526
527 print("\n=== TEST RESULTS (Random Forest, using validation KS threshold) ===")
528 print(f"Test ROC-AUC: {test_auc:.4f}")
529 print(f"Test PR-AUC : {test_prauc:.4f}")
530 print(f"Test KS : {ks_t:.4f}")
531 print("\nTest Confusion Matrix:")
532 print(confusion_matrix(y_test, test_pred))
533 print("\nTest Classification Report:")
534 print(classification_report(y_test, test_pred, digits=4))
535
536
537 # ===== PLOTS: ROC / PR / KS on TEST for 4 models (Logit
    L1, XGB, CatBoost, RF) =====
538 # Collect test-set probabilities
539 probas = {
540     "L1-Logit": best_model_final.predict_proba(X_test)[: , 1],
541     "XGBoost": best_xgb_final.predict_proba(X_test)[: , 1],
542     "CatBoost": cb_final.predict_proba(X_test_cb)[: , 1],
543     "RandomForest": best_rf_final.predict_proba(X_test_rf)[: , 1],
544 }
545
546 # Summary table: AUC / PR-AUC / KS (max TPR-FPR)
547 summary_rows = []
548 ks_curves = {}
549 for name, p in probas.items():
550     fpr, tpr, thr = roc_curve(y_test, p)
551     ks_vals = tpr - fpr
552     ks = float(np.max(ks_vals))
553     thr_ks = float(thr[np.argmax(ks_vals)])
554     auc = float(roc_auc_score(y_test, p))
555     pr_auc = float(average_precision_score(y_test, p))
556     summary_rows.append([name, auc, pr_auc, ks, thr_ks])
557     ks_curves[name] = (thr, ks_vals)
558
559 summary = pd.DataFrame(summary_rows, columns=["Model", "ROC-AUC", "PR-AUC", "KS",
    "KS_threshold"]) \
560     .sort_values("ROC-AUC", ascending=False)
561 print("\n=== TEST summary ===")
562 print(summary.to_string(index=False))
563
564 # ROC curve
565 plt.figure()
566 for name, p in probas.items():
567     fpr, tpr, _ = roc_curve(y_test, p)
568     plt.plot(fpr, tpr, label=f'{name} (AUC={roc_auc_score(y_test, p):.3f})')
569 plt.plot([0, 1], [0, 1], linestyle="--")
570 plt.xlabel("False Positive Rate")
571 plt.ylabel("True Positive Rate")
572 plt.title("ROC Curves on TEST")
573 plt.legend()
574 plt.tight_layout()
575 plt.savefig("roc_test.png", dpi=200)
576 plt.show()
577
578 # PR curve
579 plt.figure()
580 for name, p in probas.items():
581     prec, rec, _ = precision_recall_curve(y_test, p)
582     plt.plot(rec, prec, label=f'{name} (PR-AUC={average_precision_score(y_test, p):.3f})')
583 plt.xlabel("Recall")
584 plt.ylabel("Precision")
585 plt.title("Precision-Recall Curves on TEST")
```

```

586 plt.legend()
587 plt.tight_layout()
588 plt.savefig("pr_test.png", dpi=200)
589 plt.show()
590
591 # KS curve
592 plt.figure()
593 for name, (thr, ks_vals) in ks_curves.items():
594     plt.plot(thr, ks_vals, label=f"{name}")
595 plt.xlabel("Threshold")
596 plt.ylabel("KS = TPR - FPR")
597 plt.title("KS Curves on TEST")
598 plt.legend()
599 plt.tight_layout()
600 plt.savefig("ks_test.png", dpi=200)
601 plt.show()
602
603 # ===== Portfolio VaR & ES on TEST (Monte Carlo,
    independent defaults) =====
604
605 ALPHAS = [0.95, 0.975, 0.99]
606 NSIMS = 8000
607
608 N_test = len(y_test)
609 EAD_VEC = np.ones(N_test, dtype=float)
610 LGD_SCALAR = 1.0
611 W = EAD_VEC * LGD_SCALAR
612
613 def simulate_portfolio_losses(p_vec, w_vec, nsims=NSIMS, seed=SEED):
614     """Simulate total portfolio loss nsims times for Bernoulli defaults with probs p_vec."""
615     rng = np.random.default_rng(seed)
616     n = p_vec.shape[0]
617     out = np.empty(nsims, dtype=float)
618     for s in range(nsims):
619         defaults = rng.random(n) < p_vec
620         out[s] = float(np.dot(defaults, w_vec))
621     return out
622
623 def var_es_from_losses(losses, alpha):
624     """Empirical VaR and ES from a vector of simulated losses."""
625     var = float(np.quantile(losses, alpha, method="higher"))
626     es = float(losses[losses >= var].mean())
627     return var, es
628
629 rows = []
630 for name, p in probas.items():
631     p = np.asarray(p, dtype=float)
632     seed_model = SEED + int.from_bytes(
633         hashlib.sha256(f"{SEED}:{name}".encode("utf-8")).digest()[:4],
634         "little"
635     )
636
637     losses = simulate_portfolio_losses(p, W, nsims=NSIMS, seed=seed_model)
638     meanL = float(losses.mean())
639     stdL = float(losses.std(ddof=1))
640     row = {
641         "Model": name,
642         "MeanLoss": meanL,
643         "StdLoss": stdL,
644         "MeanLossRate": meanL / np.sum(EAD_VEC)
645     }
646     for a in ALPHAS:
647         v, e = var_es_from_losses(losses, a)
648         row[f"VaR@{int(a*100)}"] = v
649         row[f"ES@{int(a*100)}"] = e
650     row[f"VaRate@{int(a*100)}"] = v / np.sum(EAD_VEC)

```

Project code.py — generated 2025-08-15 17:05:47

```
651     row[f"ESrate@{int(a*100)}"] = e / np.sum(EAD_VEC)
652     rows.append(row)
653
654 var_es_df = pd.DataFrame(rows).sort_values(f"VaR@{int(ALPHAS[-1]*100)}", ascending=False)
655
656 print("\n=== TEST Portfolio VaR & ES (counts and rates; assumes indep. defaults, EAD=1,
      LGD=1) ===")
657 cols_order = ["Model", "MeanLoss", "StdLoss",
658               *(f for a in ALPHAS for f in (f"VaR@{int(a*100)}", f"ES@{int(a*100)}")),
659               "MeanLossRate",
660               *(f for a in ALPHAS for f in (f"VaRate@{int(a*100)}",
        f"ESrate@{int(a*100)}"))]
661 print(var_es_df[cols_order].to_string(index=False))
```